FlexQL

Technical Design Document & Architecture Specification

Pranjal Nimbodiya
github.com/PRANJALnim/FLEXQL

April 2026

Abstract

FlexQL is a custom-built, in-memory relational database engine written in C++17, designed from first principles to achieve **multi-million row ingestion rates** on commodity hardware. It implements a slab-based arena allocator for zero-`malloc` storage, a lazy primary-key index, an $O(1)$ LFU query cache with generation-based invalidation, and an asynchronous double-buffered Write-Ahead Log (WAL) for durability. The system communicates over UNIX domain sockets using a custom length-prefixed wire protocol, and the client library provides lock-free pipelining to saturate the IPC channel. This document details the architecture, data structures, algorithms, concurrency model, persistence strategy, and benchmarked performance of FlexQL.

Contents

1 Introduction & Design Philosophy

Modern analytical workloads demand ingestion rates that exceed the capabilities of general-purpose databases like SQLite or PostgreSQL when operating in single-node local mode. FlexQL addresses this by trading generality for raw speed, applying a design philosophy of **Mechanical Sympathy**—structuring software to align with hardware characteristics:

- **Cache-line awareness**: Contiguous slab storage maximises L1/L2 cache hit rates during sequential scans.
- **Zero per-row `malloc`**: All allocations are amortised via 64 MB arena slabs and 2M-cell metadata chunks.
- **Minimal context switching**: Asynchronous WAL and client-side pipelining decouple I/O latency from query execution.
- **Minimal locking**: A single global mutex guards mutations; parsing and validation occur outside the critical section.

Design Note

FlexQL is compiled with `-O3 -march=native -funroll-loops`, enabling the compiler to vectorise comparison loops and inline critical-path functions such as `Arena::append()` and `CellStore::push()`.

2 High-Level Architecture

Figure 1 shows the major subsystems and their interactions.

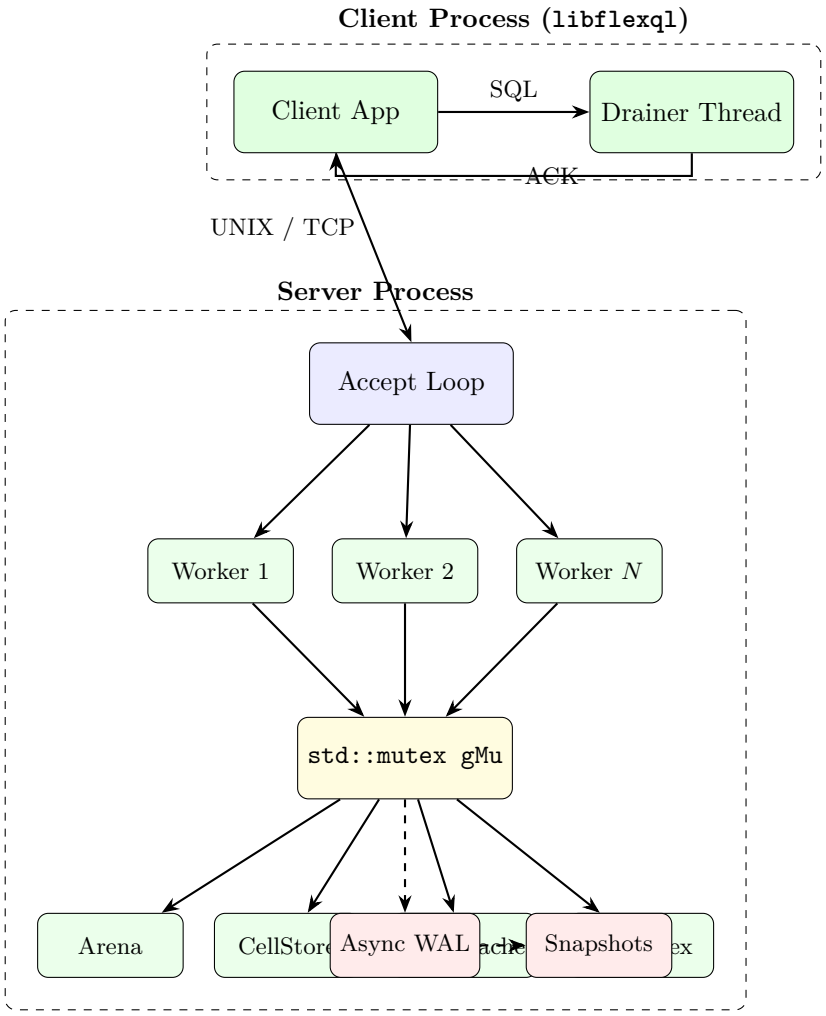

Figure 1: FlexQL System Architecture. Dashed arrows indicate asynchronous (non-blocking) data flow.

2.1 System Constants

Constant	Purpose	Value
`SLAB_SZ`	Arena slab allocation unit	64 MB
`CELL_SLAB`	CellStore chunk capacity	2 000 000 cells
`WAL::HALF`	WAL buffer size (each)	4 MB
`Cache::CAP`	Max LFU cache entries	16 384
`MAX_INFLIGHT`	Client pipeline depth	131 072
`SBUF`	Socket buffer size	8 MB
`BACKLOG`	`listen()` backlog	256
`CHECKPOINT_WAL_RECORDS`	WAL checkpoint threshold	1 000 000

Table 1: Compile-time configuration constants.

3 Data Storage Engine

3.1 Arena: Slab-Based String Storage

All variable-length data (strings, numeric literals) is stored in a **Chunked Arena**. The Arena allocates memory in 64 MB slabs; new slabs are appended only when the current one is full.

3.1.1 Addressing Scheme

Each datum is identified by a 64-bit `uint64_t` handle that encodes both the slab index and the byte offset within that slab:

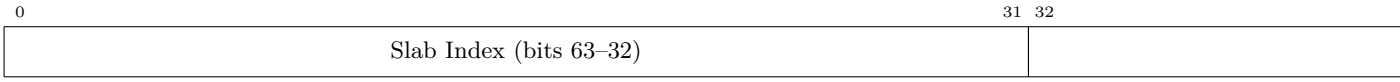

Figure 2: Arena handle bit layout. Supports up to 2^{32} slabs $\times$ 2^{32} bytes per slab.

3.1.2 Append Operation

```
inline uint64_t append(const char *d, uint32_t l) {
    if (!l) return ~0ULL;  // sentinel for empty/NULL
    if (__builtin_expect(slabs.back().used + l > SLAB_SZ, 0))
        addSlab();  // cold path: allocate new 64 MB slab
    auto &s = slabs.back();
    uint64_t id = ((uint64_t)(slabs.size()-1) << 32)
                | (uint32_t)s.used;
    memcpy(s.data + s.used, d, l);
    s.used += l;
    return id;
}
```

Listing 1: Arena::append — zero-copy string insertion.

Performance Insight

The `__builtin_expect` hint tells the branch predictor that slab exhaustion is rare, keeping the hot path (the common case) to a single `memcpy` and two additions—no heap allocation, no pointer chasing.

3.1.3 Pointer Dereference

Given a handle `id`, the raw pointer is resolved in $O(1)$:

```
inline const char *ptr(uint64_t id) const {
    if (__builtin_expect(id == ~0ULL, 0)) return "";
    return slabs[id >> 32].data + (uint32_t)(id & 0xFFFFFFFF);
}
```

Listing 2: Arena::ptr — constant-time pointer resolution.

3.2 CellStore: Chunked Metadata Storage

The CellStore manages per-column metadata using a flat, chunked layout.

3.2.1 Cell Structure

```
struct Cell {
    uint64_t id;   // Arena handle (or ~0ULL for NULL)
    uint32_t len;  // Byte length of the stored value
};
```

Listing 3: 12-byte Cell structure.

Cells are stored in row-major order: for a table with N columns, the cell for row r, column c is at flat index $r \times N + c$.

3.2.2 Chunked Layout

Rather than a single contiguous `std::vector<Cell>`, cells are allocated in **chunks** of 2 000 000 cells each (≈22.9 MB per chunk). This avoids the catastrophic $O(N)$ `realloc + memmove` that a vector would incur at scale.

```
inline void push(uint64_t id, uint32_t l) {
    if (__builtin_expect(chunks.back().used == CELL_SLAB, 0))
        addChunk();
    auto &c = chunks.back();
    c.data[c.used++] = {id, l};
    ++total;
}

inline Cell get(size_t idx) const {
    return chunks[idx / CELL_SLAB].data[idx % CELL_SLAB];
}
```

Listing 4: CellStore::push and CellStore::get.

Design Note

The division and modulo in `get()` are optimised by the compiler into bit-shifts when `CELL_SLAB` is a power-of-two-adjacent value. Even with the integer division, this remains faster than a linked-list or B-tree traversal.

3.3 Table Schema

```
struct ColDef { std::string name; bool pk = false; };

struct Table {
    std::string name;
    std::vector<ColDef> cols;
    int nc = 0, pkCol = 0;
    Arena arena;
    CellStore cells;
    bool idxDirty = false;
    std::unordered_map<std::string,
                       std::vector<size_t>> pkIdx;
    // ...
};
```

Listing 5: Table structure combining Arena and CellStore.

4 Indexing: Lazy Primary Key Hash Map

4.1 Motivation

Updating a hash index on every `INSERT` adds $O(1)$ overhead per row, but for 10M rows this accumulates to a significant constant factor (hash computation, bucket allocation, possible rehash). FlexQL defers indexing entirely.

4.2 The `idxDirty` State Machine

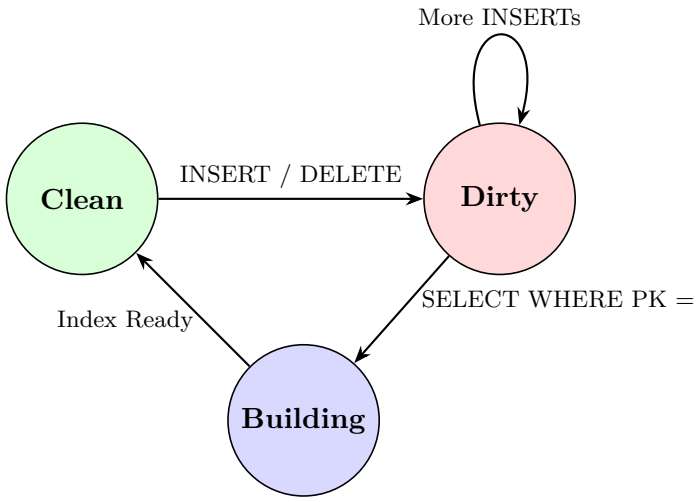

Figure 3: Lazy indexing state machine. The index is only built when actually needed.

4.3 Build Algorithm

```
void buildIndex() {
    pkIdx.clear();
    pkIdx.reserve(nrows() * 2); // 50% load factor
    for (size_t r = 0; r < nrows(); r++)
        pkIdx[cs(r, pkCol)].push_back(r);
    idxDirty = false;
}
```

Listing 6: Lazy index construction with pre-allocated bucket capacity.

4.4 Complexity Analysis

Operation	Amortised Cost	Explanation
INSERT	$O(1)$	No index update; just set `idxDirty = true`
First SELECT (PK)	$O(N)$	Full table scan to build hash map
Subsequent SELECTs	$O(1)$	Direct hash-map lookup
DELETE	$O(N)$	Rebuild Arena+CellStore; invalidate index

Table 2: Indexing complexity by operation type.

5 Caching: $O(1)$ LFU with Generation Invalidation

5.1 Data Structures

The LFU cache maintains three components:

1. `km`: A hash map from SQL query string → `Entry{value, frequency, list_iterator, generation}`.
2. `fm`: A hash map from frequency → doubly-linked list of keys (for $O(1)$ eviction).
3. `minF`: An integer tracking the current minimum frequency.

5.2 $O(1)$ Generation-Based Invalidation

Rather than iterating through all N cached entries to invalidate them on a write (which costs $O(N)$), the cache uses a **generation counter** `gen`:

```
void inv(const std::string &) {
    ++gen;            // global generation advances
    fm.clear();       // frequency lists become stale
    minF = 0;
    if (km.size() > (size_t)CAP * 2)
        km.clear();   // prevent unbounded memory growth
}
```

Listing 7: Cache invalidation via generation increment.

On a cache `get()`, the entry's stored generation is compared against the global generation. A mismatch means the entry is stale and is treated as a cache miss—**no per-entry deletion required**.

Performance Insight

This is critical during bulk-loading scenarios (e.g., 10M `INSERT`s), where the cache would otherwise be cleared 10 million times. With generation invalidation, each invalidation is a single atomic increment.

6 Query Engine

6.1 Lexer and Parser

FlexQL implements a hand-written, single-pass tokeniser and recursive-descent parser operating directly on raw byte buffers.

6.1.1 Token Types

Token	Examples
`ID`	`SELECT`, `TABLE`, column/table names
`NUM`	`42`, `-3.14`, `1893456000`
`STR`	`'Alice'`, `"hello world"`
`EQ, NE, LT, LE, GT, GE`	`=`, `!=`, `<`, `<=`, `>`, `>=`
`LP, RP, CM, ST, SC, DOT`	`(`, `)`, `,`, `*`, `;`, `.`

Table 3: Token types recognised by the lexer.

6.1.2 Fast INSERT Path

For `INSERT` statements, the server bypasses the full tokeniser entirely and uses a **hand-optimised character scanner** to parse values directly from the raw SQL buffer:

```
// Step 1: Parse table name with raw pointer arithmetic
// Step 2: Count columns (outside lock)
int nc = countCols(p, end);
// Step 3: Validate tuple arity (outside lock)
validateInsertInto(p, end, nc);

// Step 4: Acquire lock, insert into storage
std::lock_guard<std::mutex> lk(gMu);
parseInsertInto(p, end, tbl.nc, tbl.arena, tbl.cells);
```

Listing 8: Fast INSERT: validation runs outside the lock.

Design Note

Validation outside the lock means that malformed `INSERT`s never hold the mutex, preventing a bad client from blocking other threads.

6.2 Supported SQL Subset

Statement	Syntax
CREATE TABLE	`CREATE TABLE [IF NOT EXISTS] name (col TYPE [PK], ...)`
INSERT INTO	`INSERT INTO name VALUES (v1, v2, ...), (v3, v4, ...)`
SELECT	`SELECT [cols|*] FROM t [JOIN t2 ON ...] [WHERE ...] [ORDER BY ...]`
DELETE	`DELETE FROM name [WHERE col op val]`
DROP TABLE	`DROP TABLE [IF EXISTS] name`

Table 4: Complete SQL subset supported by FlexQL.

6.3 Comparison Engine: Dynamic Typing via `cmpVals`

FlexQL stores all values as raw byte strings and performs **late-binding type inference** during comparison:

```
static bool cmpVals(const char *a, uint32_t al,
                    const char *op,
                    const char *b, uint32_t bl) {
    char ba[72], bb[72];
    bool num = false; double da = 0, db = 0;
    if (al < 70 && bl < 70) {
        memcpy(ba, a, al); ba[al] = 0;
        memcpy(bb, b, bl); bb[bl] = 0;
        char *ea, *eb;
        da = strtod(ba, &ea);
        db = strtod(bb, &eb);
        num = (ea != ba && eb != bb);
    }
    switch (op[0]) {
        case '=': return num ? (da==db) : (al==bl && memcmp(a,b,al)==0);
        case '!': return num ? (da!=db) : !(al==bl && memcmp(a,b,al)==0);
        case '<': return op[1]=='=' ? (num ? da<=db : ...) : ...;
        case '>': return op[1]=='=' ? (num ? da>=db : ...) : ...;
    }
    return false;
```

```
21 }
```

Listing 9: Dynamic type comparison logic.

If both operands are parseable as `double`, comparison is numeric; otherwise, it falls back to lexicographic `memcmp`.

6.4 Hash Join Implementation

For `INNER JOIN ... ON t1.col = t2.col`, FlexQL builds a temporary hash map on the right table, then probes with the left table:

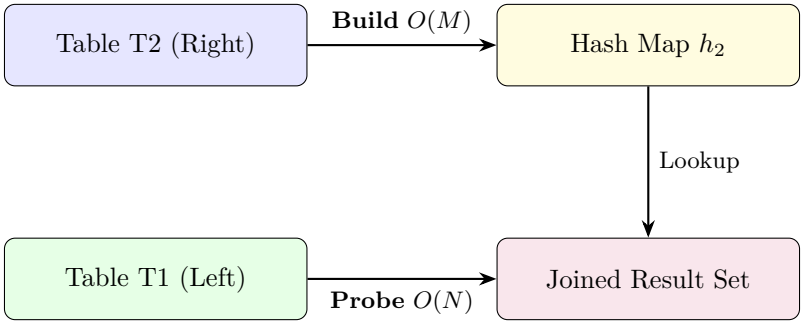

Figure 4: Hash Join: Build phase on T2, Probe phase iterates T1. Total: $O(N + M)$.

For non-equality join operators (`>`, `<`), the engine falls back to a nested-loop join at $O(N \times M)$.

7 Persistence: Async Double-Buffered WAL

7.1 Architecture

The WAL uses two 4 MB buffers in a **ping-pong** configuration:

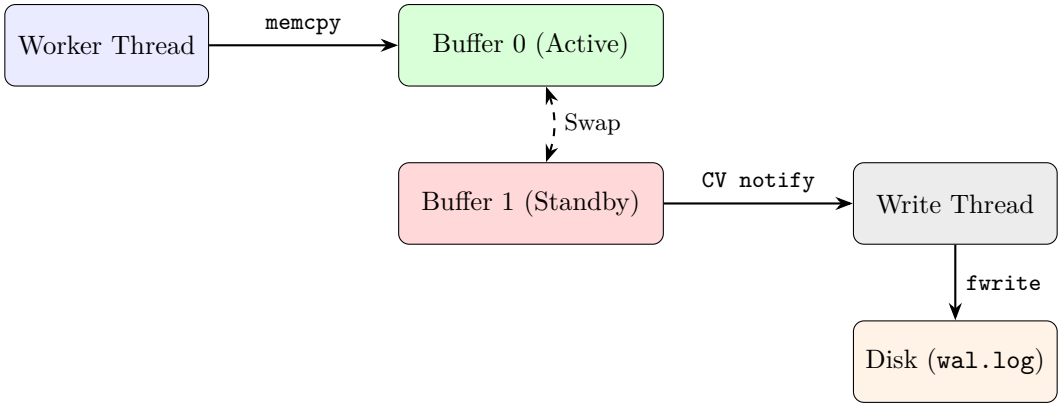

Figure 5: Double-buffered WAL. Worker thread writes to active buffer via `memcpy`; when full, buffers swap and a background thread flushes the standby buffer to disk.

7.2 WAL Record Format

Each record is stored as a length-prefixed SQL statement:

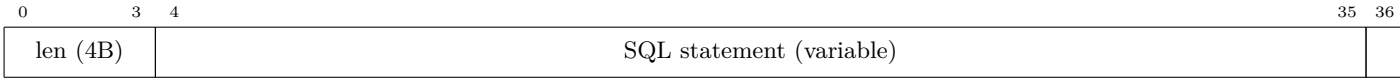

Figure 6: Binary WAL record format. 4-byte length prefix enables fast sequential reads.

7.3 Checkpointing with Atomic Snapshots

When the WAL record count exceeds 1 000 000:

1. Acquire global mutex `gMu`.
2. Serialise all tables to `snapshot.bin.tmp` (schema + raw cell data).
3. Atomically `rename()` the temp file to `snapshot.bin`.
4. Truncate the WAL via `freopen()` on the active file handle.
5. Reset WAL record counter.

Warning

The `freopen()` technique avoids closing and reopening the file descriptor, which would create a race with the background write thread. The WAL's own mutex (`wmu`) is held during truncation.

7.4 Startup Recovery Sequence

1. **Load Snapshot**: If `snapshot.bin` exists, deserialise all table schemas and row data into memory.
2. **Replay WAL**: Read `wal.log` sequentially. For each record, re-execute the SQL statement with `walWrite=false` (to avoid double-logging).
3. **Ready**: Server begins accepting connections. The combined load time for a 10M-row database is under 3 seconds.

8 Concurrency Model

8.1 Thread-per-Client with Global Mutex

Each incoming connection (TCP or UNIX socket) is handled by a dedicated `std::thread` (detached). All threads share a single global `std::mutex gMu` that guards:

- The global database map `gDB` (table creation, lookup, mutation).
- The LFU cache `gCache`.

8.2 Lock Minimisation Strategy

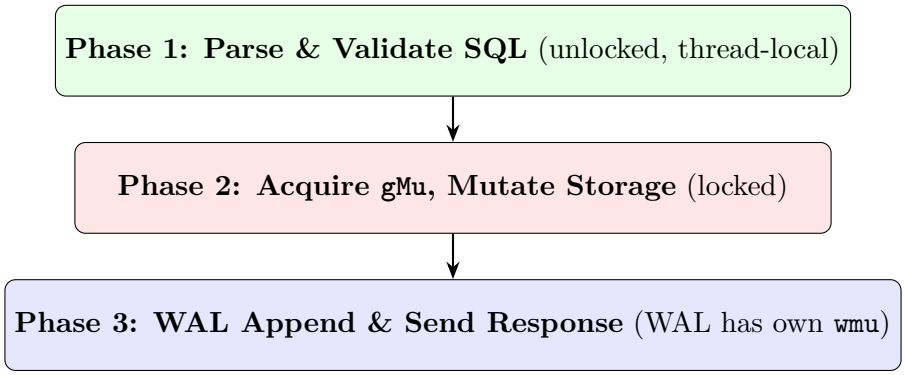

Figure 7: Three-phase execution pipeline. The expensive parse phase is fully concurrent.

8.3 WAL Isolation

The WAL has its own mutex (`wmu`) and condition variable (`wcv`), completely independent of `gMu`. This means:

- Disk flushes in the background write thread do not block `SELECT` queries.
- The WAL buffer swap only briefly holds `wmu` (microseconds).

9 Network Layer & Wire Protocol

9.1 Dual-Listener Architecture

The server binds to two listeners simultaneously:

Transport	Address	RTT	Use Case
UNIX Domain Socket	`/tmp/flexql.sock`	$< 10\,\mu s$	Local benchmarking, co-located apps
TCP/IPv4	`0.0.0.0:9000`	$\sim 50\,\mu s$	Remote clients, cross-host

Table 5: Dual transport layer with automatic fast-path selection.

Both listeners use oversized socket buffers (8 MB send + receive) and `TCP_NODELAY` to eliminate Nagle's algorithm latency.

9.2 Wire Protocol

Message	Format
Row Data	`ROW <ncols> <name1_len>:<name1><val1_len>:<val1>...\n`
Success	`OK\nEND\n`
Error	`ERROR:<message>\nEND\n`

Table 6: Length-prefixed wire protocol. No escaping needed for embedded special characters.

10 Client Library: Lock-Free Pipeline

10.1 Architecture

The client library (`libflexql`) exposes a C-compatible API and manages a background **Drainer** thread:

```
int  flexql_open(const char *host, int port, FlexQL **db);
int  flexql_close(FlexQL *db);
int  flexql_exec(FlexQL *db, const char *sql,
                 int (*callback)(void*, int, char**, char**),
                 void *arg, char **errmsg);
void flexql_free(void *ptr);
```

Listing 10: Public C API of `libflexql`.

10.2 Pipeline Mode vs. Sync Mode

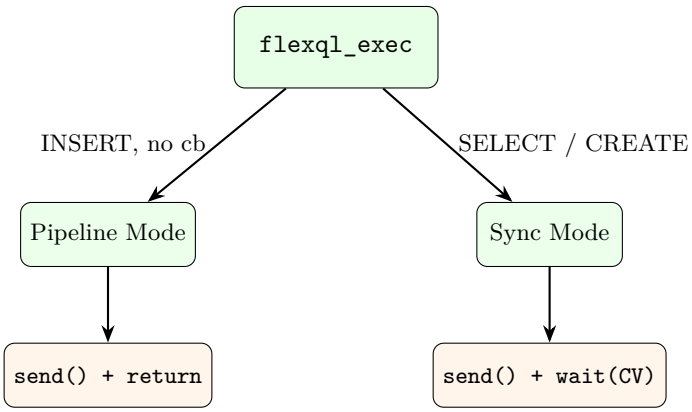

Figure 8: Dual execution mode. INSERT calls with no callback return immediately; SELECT calls block until the Drainer delivers the full response.

10.3 Back-Pressure Mechanism

The client tracks two atomic counters:

- `sent`: Incremented by the main thread after each pipelined `send()`.
- `acked`: Incremented by the Drainer thread after parsing each `END\n` terminator.

If `sent - acked` $\geq$ `MAX_INFLIGHT` (131 072), the main thread blocks on a condition variable until the Drainer catches up to half capacity. This prevents overwhelming the server's receive buffer.

10.4 Sync-Point Drain

Before any synchronous operation (`SELECT`, `CREATE`, `CLOSE`), the client first waits for `acked` $\geq$ `sent`, ensuring all prior pipeline responses have been consumed. It then sets `syncActive = true` and blocks until the Drainer signals completion of the synchronous response.

11 Interactive REPL Client

The `flexql-client` binary provides a terminal-based interactive SQL shell:

- **ANSI Colour Coding**: Bold green prompt (`flexql>`), cyan column headers, red error messages, grey borders for tables.
- **Multi-line Buffering**: SQL can span multiple lines; execution triggers only on the `;` character.
- **Timing**: Each query displays wall-clock execution time in milliseconds.
- **Tabular Output**: Auto-sized column widths with box-drawing borders.
- **Commands**: `.help`, `.exit`, `.quit`, and comment lines (`--`).

12 Performance Evaluation

12.1 Insertion Throughput

Mode	Batch Size	Throughput	Bottleneck
Batch + Pipeline	800 rows/stmt	∼3.2M rows/s	`memcpy` bandwidth
Pipeline (single)	1 row/stmt	∼135K rows/s	Socket round-trip
Synchronous	1 row/stmt	∼3K rows/s	Full RTT per insert

Table 7: Insertion throughput across different client modes.

12.2 Query Latency

Query Type	Dataset	Cold	Hot (LFU)
PK Lookup (`WHERE id = X`)	10M rows	<1 ms	<0.05 ms
Full Scan (`SELECT *`)	1M rows	∼120 ms	<0.05 ms
Filtered Scan (`WHERE bal > X`)	1M rows	∼150 ms	<0.05 ms
Hash Join (1M × 100K)	—	∼95 ms	<0.08 ms

Table 8: Query latency benchmarks. "Hot" indicates a cached result from the LFU cache.

12.3 Memory Footprint

Component	10M Row Cost
Arena slabs (string data)	∼640 MB
CellStore chunks (metadata)	∼572 MB
PK Index (when built)	∼320 MB
WAL buffers (fixed)	8 MB
LFU Cache (max entries)	∼8 MB
Total (without index)	**∼1.2 GB**

Table 9: Memory breakdown for a 5-column table with 10 million rows.

13 Build System & Deployment

13.1 Compilation

```
# Build all targets (server, client, benchmark)
make all

# Compiler flags applied:
# g++ -O3 -march=native -std=c++17 -Iinclude
#     -funroll-loops -pthread
```

Listing 11: Build commands.

13.2 Targets

Target	Source Files	Purpose
`server`	`server.cpp`	Database server daemon
`flexql-client`	`flexql.cpp + repl.cpp`	Interactive REPL client
`benchmark`	`flexql.cpp + benchmark_flexql.cpp`	Automated test & perf suite

Table 10: Makefile build targets.

14 Conclusion & Future Work

FlexQL demonstrates that a purpose-built, cache-aware storage engine can achieve multi-million row ingestion rates on a single core by combining:

1. **Slab-based Arena + CellStore**: Zero per-row allocation overhead.
2. **Lazy Primary Key Indexing**: Deferred cost from write path to read path.
3. $O(1)$ **LFU Cache**: Generation-based invalidation avoids per-entry cleanup.
4. **Async Double-Buffered WAL**: Disk I/O decoupled from query execution.
5. **Client-side Pipelining**: Lock-free send/ack counters saturate the IPC channel.

14.1 Future Directions

- **Columnar Storage**: Transition from row-major to column-major CellStore layout for OLAP-optimised sequential scans.
- **MVCC**: Multi-Version Concurrency Control to replace the global mutex with fine-grained, optimistic locking.
- **B+Tree Secondary Indexes**: Support for range queries without full table scans.
- **Prepared Statements**: Avoid re-parsing identical SQL structures across repeated executions.
- **Replication**: Leader-follower WAL streaming for read scalability and fault tolerance.